

Warehouse Inspector

Automated tile inspection with a **U-Net**

From a Colab notebook to the warehouse floor

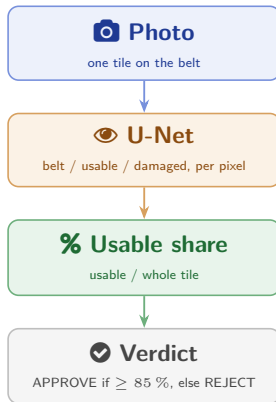
What the Warehouse Inspector is

The problem

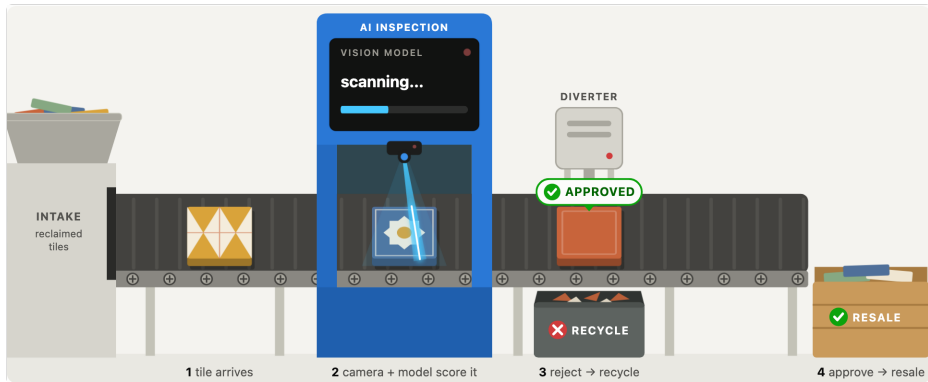
A recycling warehouse receives reclaimed floor tiles **by the thousand**. Today a person checks every one: is it cracked, chipped, still sellable? It is slow, tiring and two inspectors rarely agree on the borderline tiles.

What we built

An **inspection station**: each tile rides a conveyor under a camera, a **U-Net** marks the usable surface, and the station decides **sell or recycle**. People only see the tiles the model is unsure about.



The inspection line



Tiles arrive from intake, the camera and the **vision model** score each one, and a diverter sends it to **resale** or **recycling**.

The notebook opens with this animation: it is the machine the students are about to build the brain for.

Two halves of the same thing

The exercise — what students build

A Colab notebook. Students write the dataset, the U-Net, three decision heads, run a receptive-field experiment and try transfer learning.

Live panels grade every step against targets.

 `notebook.ipynb`

The platform — what it becomes

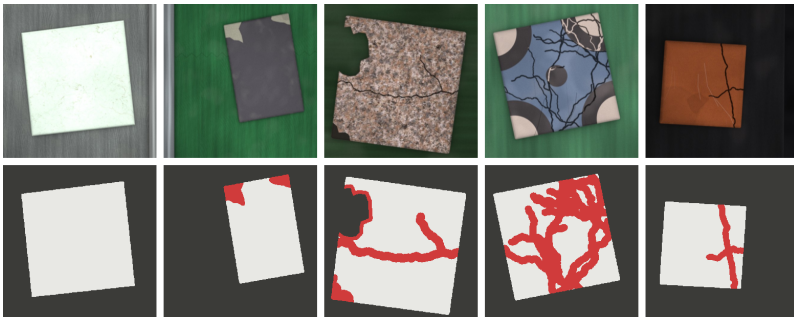
The same model inside the warehouse's web app: upload photos or use a phone camera, review queue, lots, dashboard.

It runs on the student's own laptop.

 **Inspection platform**

One file connects them: `platform_model.pt`. The notebook exports it in §11, the platform loads it under *Settings*. What the platform shows is exactly what the student trained.

The data: one tile per photo



pristine marble

100 % ✓

5 000

photos with pixel-exact masks

chipped corners

92 % ✓

crack + chip

84.6 % ✗

55 %

approved; 37 % are pristine

crack network

54 % ✗

scratches are fine

89 % ✓

6 × 5

materials × belts, CC0 textures

Three classes, one rule

Every pixel is one of three classes

■	0 belt	not part of the tile
□	1 usable	surface that can be sold
■	2 damaged	cracks, chips, missing pieces + a 3% cutting margin

The hard part

Stains, dirt and scratches are **not** damage: the tile can be cleaned. A thin crack and a dark vein of marble look alike up close.

The decision rule

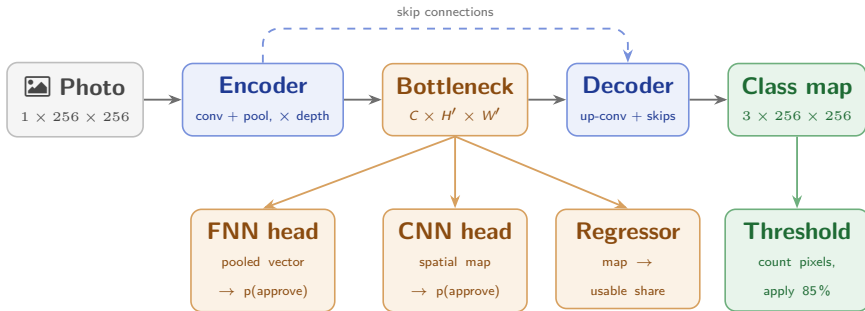
$$\text{usable share} = \frac{\text{usable pixels}}{\text{usable} + \text{damaged pixels}}$$

- $\geq 85\%$ → **APPROVE**: sell it, cut down if needed
- $< 85\%$ → **REJECT**: recycle it

Measured on the tile, not on the photo: a small tile far from the camera is judged like a big one.

One number, `data.USABLE_THRESHOLD`, used by the notebook and the platform.

The pipeline: four ways to reach a verdict



Segment, then count

The U-Net draws the map; the rule does the rest. Transparent: you can see why a tile was rejected.

Decide from the latent space

Small heads on the frozen bottleneck. What survives the compression, and does throwing away *where* hurt?

What students build, step by step

§	Step	They write	The question it answers
2	Dataset	a Dataset: photo, class map, label 	can we feed the model correctly?
3	First U-Net	encode / decode, the training loop 	is segment-and-count enough?
4–5	FNN and CNN heads	two classifiers on the bottleneck 	does the layout of the damage matter?
6–7	Evaluation	— (scoreboard, inspector, game)	which tiles fail, and why?
8	Receptive field	— (depth-2/3/4 experiment)	why cracks look like stains
9	Challenge	improved U-Net, heads, a regressor 	can we beat every target?
10	Transfer learning	the decoder of a ResNet-18 U-Net 	do ImageNet features help?
11	Deploy	— (export + download)	does it work in the product?

 = one of the 11 TODO cells. Every section ends with a  *Why this step?* note that hands its answer to the next.

Why the first model fails: the receptive field

What a pixel can see

Each output pixel of a CNN depends on a limited patch of the input, its **receptive field**. If the patch is smaller than the pattern, a crack and a dark stain look the same.

The experiment (§8)

Train the same U-Net at **depth 2, 3 and 4**. Measure the theoretical and the *effective* receptive field (input gradients), and click a spot on a tile to see what each model sees there.

Threshold accuracy on the test set

Basic U-Net (§3)	86.3 %
DepthUNet, depth 2	92.4 %
DepthUNet, depth 3	92.5 %
DepthUNet, depth 4	92.8 %

The lesson

Depth sharpens the masks, but the verdict saturates. The rest has to come from a better architecture, training and pretrained features.

The journey in numbers

86.3 %

basic U-Net (§3)

97.3 %

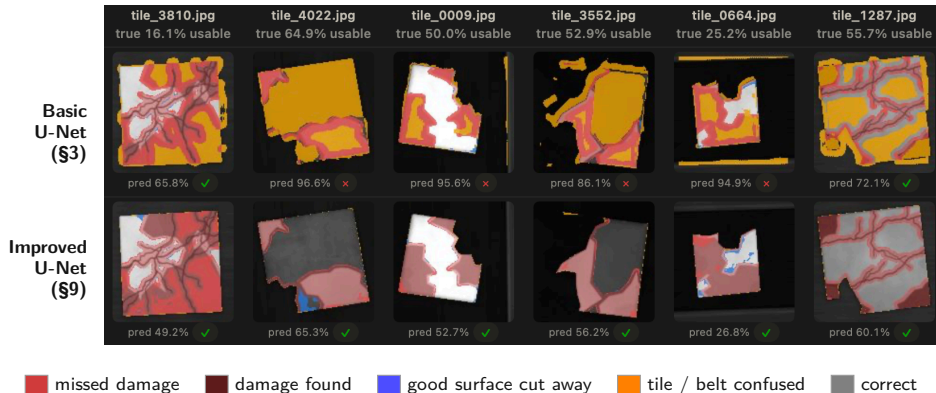
improved U-Net (§9)
threshold-rule accuracy on 750 test tiles

98.5 %

pretrained ResNet-18 (§10)

	mean IoU	usable-share error	CNN head	regressor
Basic U-Net	0.587	7.5 pp	70.8 %	—
Improved U-Net	0.935	1.86 pp	93.1 %	91.7 %
ResNet-18 U-Net	0.958	0.92 pp	—	94.1 %
Target (§9)	≥ 0.90	≤ 2.5 pp	≥ 92 %	≥ 90 %

Same tiles, different models



The six test tiles where the basic model misses the most damage, from a real Colab run. The basic U-Net approves four of them; the improved one gets all six right.

An interactive notebook

Train

Live dashboard

loss and accuracy per epoch,
against the target

Architecture diagram

every block with its shape

Evaluate

Scoreboard

every method against its target,
all attempts kept

Confusion matrix

click a cell to see those tiles

Inspector and galleries

error overlays, model vs. model

Explore

Threshold & cost explorer

drag the line, price the mistakes

Receptive-field visualiser

click a spot, see what each
model sees

You vs. the model

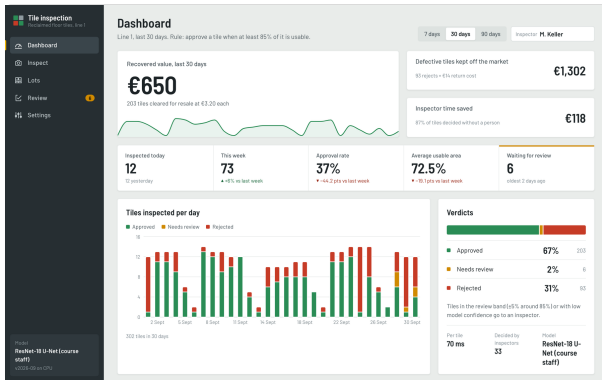
a game in Colab, scored against
the truth

Built for a course that runs over days

Plain HTML panels (they work in Colab, Safari included, and stay in the saved notebook). Plumbing cells are folded into Colab forms. Models are saved to Drive, and a ► **RESUME HERE** cell per section reloads exactly what that section needs.

The platform: what the warehouse runs

Dashboard	value recovered, approval rate, throughput, suppliers
Inspect	drop in photos or use the phone camera; watch the camera gate
Lots	one shipment: accept or reject, CSV, printable report
Review	borderline tiles for a person, least confident first
Settings	threshold, review band, prices, the model file



FastAPI + SQLite + React. No login; every decision is signed and audited.

How the pieces talk



⇌ 1 A REST request

The browser sends photos and asks for data; the server answers with JSON.

POST `/api/inspections`

GET `/api/dashboard`

GET `/api/review-queue`

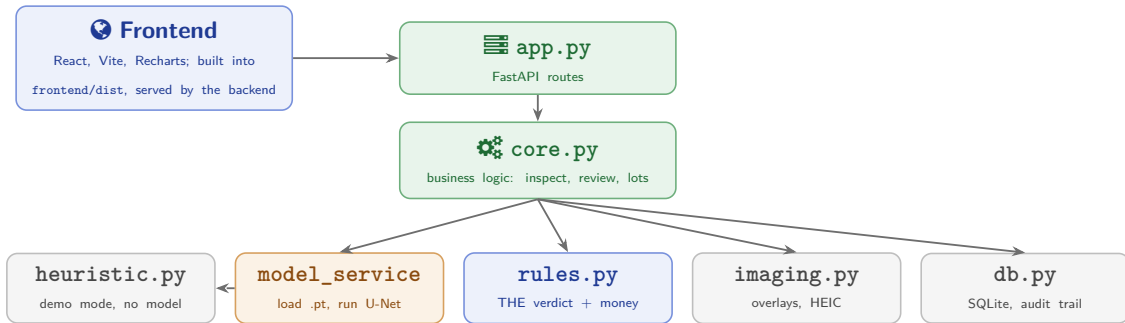
🖼️ 2 Stored images

Every photo, overlay and class mask is saved on disk and served as a file, so the database stays small and the history can be re-opened.

⚙️ 3 The model

Loaded once at start-up.
The same preprocessing as the notebook: greyscale, 256×256 , values in $[0, 1]$.

System architecture



One decision module. Nothing but `rules.py` decides a verdict, so the notebook's rule, the dashboard and the what-if in Settings can never disagree.

Every inspection stores the rule and the model version it was decided with; changing the rule never rewrites history.

One tile through the platform



Usable share	Verdict
$\geq 90\%$	APPROVE
80 – 90 %	REVIEW
$< 80\%$	REJECT

the ± 5 -point review band around 85 % (all adjustable)

Confidence

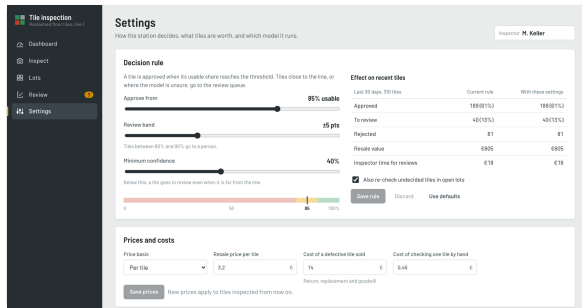
$\sqrt{\text{margin} \times \text{mask certainty}}$: far from the line *and* a crisp mask. Below 40 % a tile goes to review, whatever its share. On a laptop CPU the ResNet U-Net takes ≈ 70 ms per tile.

From accuracy to money

Not every mistake costs the same

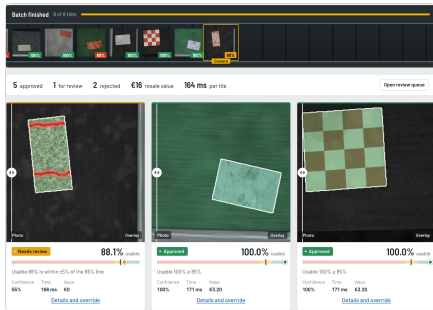
- a **false approval** ships a damaged tile: return, replacement, goodwill — **€ 14**
- a **false rejection** scraps a sellable tile — **€ 3.20**
- a **manual check** costs inspector time — **€ 0.45**

The notebook's cost explorer (§9.6) and the platform's what-if ask the same question: *where should the line and the review band sit?*



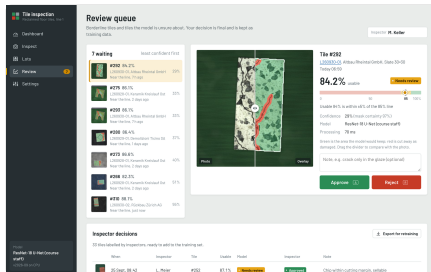
Settings: move a slider, see the last 30 days re-decided before saving.

What it looks like



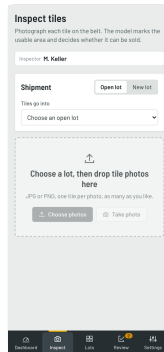
Inspect

a batch after the camera gate: overlay slider, share, verdict



Review

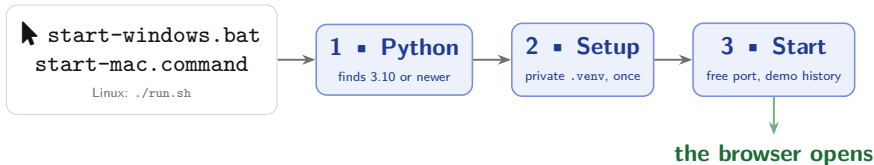
a person decides the borderline tiles; the decision becomes training data



Phone

camera capture on the same Wi-Fi

Getting it running: double-click



No Node needed

The interface ships pre-built and is served by the backend, so nobody runs `npm`.

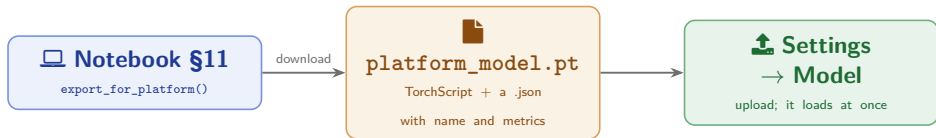
Nothing system-wide

Packages go into the platform's own `.venv`. An interrupted install is simply redone.

A model included

It starts with the staff's ResNet-18 U-Net, so the demo works before anyone has trained anything.

Your model in the platform

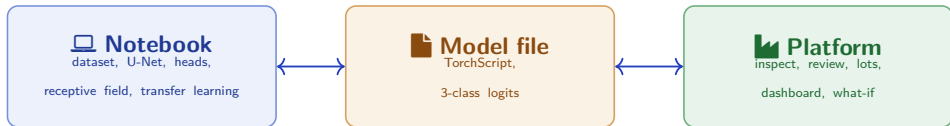


Why not the notebook checkpoint

`save_model (dill)` stores the student's classes as Python bytecode: a Colab file (Python 3.13) fails on a laptop with 3.11. Good for resuming, not for shipping.

The platform accepts any model that maps $(B, 1, 256, 256)$ in $[0, 1]$ to 3-class logits $(B, 3, 256, 256)$.

In one picture



A camera and a U-Net that **mark the usable surface, decide sell or recycle,** and **hand the hard cases to a person** — built by the students, run by the warehouse.

Notebook: `week3/project1/notebook.ipynb` • Platform: double-click `start-windows.bat` /
`start-mac.command` → `localhost:8000`

Questions?

Warehouse Inspector — automated tile inspection, from notebook to warehouse floor
